

Thermodynamic State Vector Interface & Systems Ecology Accounting in the Web of Life Simulation Architecture: Sprint 17 Technical Report

Lead Scientific Communications & Academic Outreach Agent
Web of Life Research Group
<https://github.com/pascalranoroarijaona/WebOfLife>

Sprint 17 - Thermodynamic Architecture

Abstract

As the Web of Life simulation architecture matures in Sprint 17, maintaining rigorous thermodynamic accounting across elemental cycles (Carbon, Nitrogen, Phosphorus, Water) and metabolic monads requires a unified, formal type contract. This sprint formally consolidates monadic thermodynamic process structures and boundary flux methods through the **Thermodynamic State Vector Interface** (`src/thermodynamics/types.ts`). The specification enforces strict TypeScript interfaces for internal entropy generation ($\dot{S}_{\text{gen}} \geq 0$), exergy destruction rate via the Gouy-Stodola theorem ($\dot{I} = T_0 \dot{S}_{\text{gen}}$), and multi-port boundary flux arrays. This paper outlines the theoretical foundations, interface contracts, class hierarchies, and automated verification protocols implemented to guarantee absolute adherence to the First and Second Laws of Thermodynamics across all simulated ecological monads.

1 Introduction and Systems Ecology Motivation

Ecosystems operate as open thermodynamic systems far from equilibrium, driven by continuous solar exergy inputs and governed by entropy dissipation. In silico ecosystem models frequently struggle with thermodynamic closure, often violating mass-energy conservation or ignoring internal entropy accumulation.

Sprint 17 resolves these architectural limitations by embedding fundamental thermodynamic laws directly into the type system of the Web of Life simulation engine (`src/thermodynamics/types.ts`). By defining strict contracts for state vectors, boundary fluxes, and monad processes, we ensure that every simulated ecological entity operates within unyielding thermodynamic boundaries.

2 Thermodynamic First and Second Law Principles

Every subsystem operating within the Web of Life strictly adheres to universal thermodynamic balance equations:

2.1 First Law (Energy Conservation)

$$\frac{dE_{\text{system}}}{dt} = \sum_j \dot{Q}_j - \dot{W} + \sum_{in} \dot{m}_{in} h_{in} - \sum_{out} \dot{m}_{out} h_{out} \quad (1)$$

2.2 Second Law (Entropy Balance & Clausius-Duhem Inequality)

$$\frac{dS_{\text{system}}}{dt} = \sum_j \frac{\dot{Q}_j}{T_j} + \sum_{in} \dot{m}_{in} s_{in} - \sum_{out} \dot{m}_{out} s_{out} + \dot{S}_{\text{gen}} \quad (2)$$

where the internal entropy generation rate satisfies:

$$\dot{S}_{\text{gen}} \geq 0 \quad (3)$$

2.3 Exergy Destruction Rate (Gouy-Stodola Theorem)

$$\dot{I} = T_0 \dot{S}_{\text{gen}} \geq 0 \quad (4)$$

where T_0 is the ambient reference temperature (Kelvin).

3 Interface Specifications (src/thermodynamics/types.ts)

The new type definitions establish precise data structures for state vectors, boundary fluxes, and thermodynamic metrics:

```
export type EnergyJoules = number;
export type EntropyJoulesPerKelvin = number;
export type TemperatureKelvin = number;
export type PowerWatts = number;
export type MassKilograms = number;
export type MassFluxRate = number; // kg/s

export interface IThermodynamicBoundaryFlux {
  readonly portId: string;
  readonly heatFluxWatts: PowerWatts;
  readonly boundaryTemperatureKelvin: TemperatureKelvin;
  readonly massFlowRateKgPerSec: MassFluxRate;
  readonly specificEnthalpyJoulesPerKg: number;
  readonly specificEntropyJoulesPerKgKelvin: number;
}

export interface IThermodynamicStateVector {
  readonly timestamp: number;
  readonly internalEnergy: EnergyJoules;
  readonly totalEntropy: EntropyJoulesPerKelvin;
  readonly systemTemperature: TemperatureKelvin;
  readonly ambientReferenceTemperature: TemperatureKelvin;
  readonly boundaryFluxes: readonly IThermodynamicBoundaryFlux[];
  readonly entropyGenerationRate: EntropyJoulesPerKelvin;
  readonly exergyDestructionRate: PowerWatts;
}

export interface IThermodynamicSystem {
  getStateVector(): IThermodynamicStateVector;
  validateFirstLaw(tolerance?: number): boolean;
  validateSecondLaw(): boolean;
}
```

4 Concrete Monad Implementation & Biochemical Mappings

The abstract base class `ThermodynamicMonadProcess` enforces second law validation and Gouy-Stodola consistency upon every state transition.

4.1 Biochemical Cycle Integration

1. **Carbon Cycle Photosynthesis:** Models photon exergy absorption, thermal degradation, and chemical bond storage.
2. **Water Cycle Evapotranspiration:** Incorporates latent heat of vaporization ($\Delta H \approx 2.26 \times 10^6$ J/kg) and vapor diffusion resistance.

5 Verification Protocols (`tests/sprint_017.test.ts`)

1. **Negative Entropy Rejection Test:** Asserts that assigning $\dot{S}_{\text{gen}} < 0$ throws an immediate Second Law violation error.
2. **Gouy-Stodola Consistency Check:** Verifies exact computation of $\dot{I} = T_0 \dot{S}_{\text{gen}}$ within 10^{-9} floating-point tolerance.
3. **First Law Closure Test:** Validates multi-port boundary energy balances across metabolic loops.

6 Conclusion

Sprint 17 establishes an uncompromising thermodynamic foundation for the Web of Life simulation architecture. By enforcing energy conservation, non-negative entropy generation, and exergy destruction limits at the type level, the framework guarantees physical realism across all ecological monads and biogeochemical cycles.

Official Repository: <https://github.com/pascalranoroarijaona/WebOfLife>